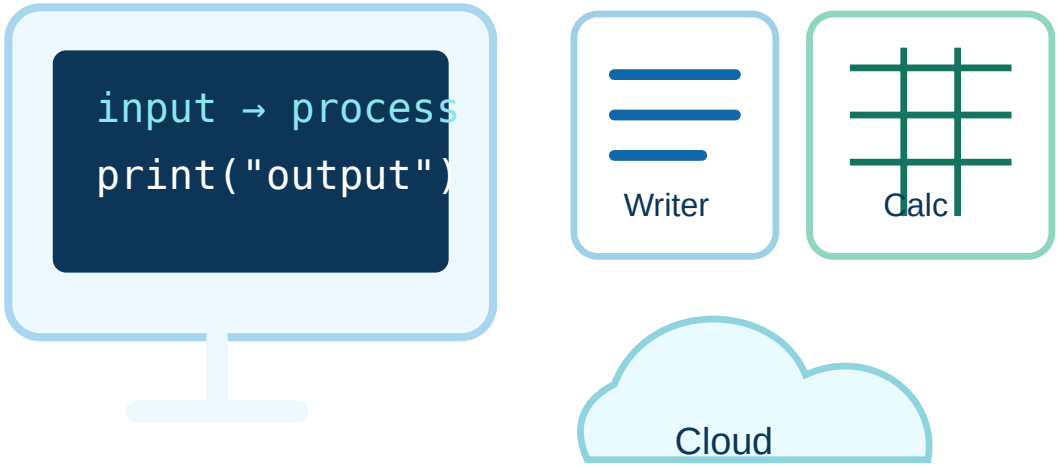
KERALA POLYTECHNIC • SEMESTER 1 • COMPLETE PRACTICAL HANDBOOK

Introduction to IT Systems Lab

A full laboratory handbook for computer fundamentals, operating systems, Internet applications, OpenOffice and cloud tools, algorithms, flowcharts and beginner Python programming—with Malayalam helpers, practical records, solved tasks, projects, troubleshooting and answer keys.

Course Code	1008
Credits	2
Category	Engineering Science
Periods	45
L:T:P	1:0:2
COs	4
Series Tests	2 × 1 hour
Author	Nandakumar.M

From computer use to Python problem solving


input → process
print("output")
Writer
Calc
Cloud

OFFICIAL COURSE STRUCTURE

Objectives, prerequisites and outcomes

Computer and Internet confidence

Recognise basic computer functions and features, use an operating system, organise files and work safely with browsers, search engines, email and web portals.

> Computer □□□□□□ safe □□□ □□□□□□□ files organise □□□□□□□□□ Internet services □□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□ □□□□□□.

Office productivity

Create documents, spreadsheets and presentations using stand-alone and cloud-based tools.

> Writer, Calc, Impress, cloud tools □□□□□□ □□□□□□□□□□ professional output □□□□□□□□□□□□.

Problem solving with Python

Design algorithms and flowcharts, then translate sequence, selection and iteration into working Python programs.

> Problem □□□□□ algorithm/flowchart □□□ plan □□□□□□ □□□□□□□□ Python code □□□□□□.

Prerequisites

- Basic school-level computer knowledge.
- Mathematics from Classes 8–10: arithmetic, percentage and comparison.
- Basic typing and logical thinking.

Study method

1. Understand the purpose and terminology.
2. Perform the practical using the exact sequence.
3. Capture observation or output evidence.
4. Write result and precautions in the record.
5. Modify the example and test again.

CO	Official outcome	Hours	Level
CO1	Utilise basic functions and features of computer, operating system and Internet applications.	9	Applying
CO2	Use stand-alone and cloud-based office tools to prepare documents, spreadsheets and presentations.	11	Applying
CO3	Develop algorithms and flowcharts for simple problems.	5	Applying

CO	Official outcome	Hours	Level
CO4	Develop Python programs to solve simple problems.	18	Applying

Hour check: 9 + 11 + 5 + 18 = 43 teaching hours. Series Test I and Series Test II add one hour each, producing the official 45 periods.

CO–PO MAPPING

Outcome relationship

Outcome	PO1	PO2	PO3	PO4	PO5	PO6	PO7
CO1	3	—	—	—	—	—	—
CO2	3	—	—	3	—	—	—
CO3	3	—	—	—	—	—	—
CO4	3	—	3	—	—	3	—

3 denotes strong mapping in the official table.

MODULE MAP

Forty-five-period learning journey

Module 1 • 9 h

Computer hardware, ports, operating system, files, commands, browsers, search, email and portals.

Module 2 • 11 h

OpenOffice Writer, Calc, Impress and cloud-based office collaboration.

Module 3 • 5 h

Algorithms and flowcharts for sequence, selection and iteration.

Module 4 • 18 h

Python input/output, expressions, decisions, loops and open-ended projects.

Before practical work

IT Laboratory Orientation

Safe equipment handling, ergonomics, digital responsibility, file organisation and practical-record discipline.

Safety, ergonomics and responsible use

Workstation posture

- ✓ Keep feet supported and shoulders relaxed.
- ✓ Place the monitor roughly an arm's length away.
- ✓ Keep the top of the screen near eye level.
- ✓ Keep wrists neutral while typing.
- ✓ Take short visual and movement breaks.

Electrical and equipment safety

- ✓ Use the instructed power sequence and UPS.
- ✓ Keep liquids away from equipment.
- ✓ Do not insert damaged cables.
- ✓ Do not open equipment without supervision.
- ✓ Report heat, smell, sparks or damaged ports immediately.

Digital safety

- ✓ Use strong unique passwords.
- ✓ Do not save passwords on shared systems.
- ✓ Scan removable drives where required.
- ✓ Avoid unknown downloads and links.
- ✓ Log out from email and cloud accounts.

□□□□□□□ □□□□□□□□□□□□□□□□: Cable force □□□□□□□□□, unknown USB/download □□□□□□□□□□□□□, shared computer-□ account logout □□□□□□□□ □□□□□□□□.

File organisation and record format

Recommended naming

`1008_M2_Writer_ProjectReport_Team03_v02.odt`

- Use course code and module.
- Use meaningful task name.
- Add team or roll number where required.
- Use version numbers instead of “final-final”.

```
Course-1008/
├── Module-1/
├── Module-2/
│   ├── Writer/
│   ├── Calc/
│   └── Impress/
├── Module-3/
├── Module-4/
│   └── Python/
├── Projects/
└── Backup/
```

Practical-record template

1. Experiment number and date
2. Aim and requirements
3. Relevant theory
4. Procedure or algorithm
5. Observation / screenshot / program
6. Input and output or result table
7. Result
8. Precautions and faculty signature

> **Evidence rule:** Screenshots must show relevant work, not passwords or private personal information.

Before starting

Create the correct folder, read the task, check equipment and name the file before entering data.

Before saving

Check content, formatting, formulas, file type and storage location.

Before submission

Open the saved file, verify output, export PDF when required, attach the correct file and keep a backup.

Searchable reference

IT Key Bank

Core terms for computer use, files, Internet, algorithms and programming.

Computer foundations

Term	Meaning	Example
Data	Raw facts such as marks, names or readings.	72, Anu, 15-06-2026
Information	Processed data that has meaning.	Class average = 72%
Hardware	Physical parts of a computer.	Keyboard, RAM, SSD
Software	Programs and instructions executed by hardware.	Operating system, Writer, Python
Firmware	Software stored in non-volatile memory that controls hardware.	UEFI/BIOS
CPU	Main processing unit that executes instructions.	Runs the Python interpreter
ALU	CPU section performing arithmetic and logic.	Addition and comparison
Control Unit	Coordinates instruction execution and data movement.	Fetch–decode–execute
Register	Very small, fast storage inside the CPU.	Current instruction/data
RAM	Temporary working memory.	Open applications
ROM	Non-volatile memory for fixed startup instructions.	Firmware storage
Cache	Fast memory near the CPU.	Frequently used data
HDD	Magnetic secondary storage with moving parts.	Large low-cost storage
SSD	Flash secondary storage without moving parts.	Fast boot
Motherboard	Main circuit board connecting components.	Contains sockets and slots
Booting	Starting the computer and loading the OS.	Power-on boot
Operating system	System software managing hardware and services.	Windows or GNU/Linux
Device driver	Software enabling OS–hardware communication.	Printer driver

Files, Internet and programming

Term	Meaning	Example
File	Named collection of data.	report.odt
Folder / directory	Container organising files and subfolders.	Course-1008/Module-2
Path	Location of a file or folder.	C:\Course-1008\report.odt
File extension	Suffix commonly indicating file type.	.odt, .ods, .odp, .py
Copy	Creates a duplicate while keeping the original.	Backup copy
Move	Changes an item's location.	Move screenshot
Browser	Application used to access web resources.	Firefox or Chrome
Search engine	Service that indexes and finds web content.	Search for Python documentation
URL	Address of a web resource.	https://example.org/page
Domain	Human-readable Internet name.	example.org
DNS	Translates domain names into IP addresses.	Finds a server

Term	Meaning	Example
HTTPS	HTTP protected by TLS encryption.	Secure transport
Attachment	A file sent with an email.	project-report.pdf
BCC	Hidden copy to recipients.	Privacy-sensitive group mail
Phishing	Fraud attempting to steal credentials.	Fake password reset
Algorithm	Finite, ordered and unambiguous solution steps.	Steps to calculate average
Pseudocode	Language-independent structured algorithm description.	READ a,b; PRINT a+b
Flowchart	Graphical representation of control flow.	Decision diamond
Sequence	Statements executed in order.	Input → calculate → output
Selection	Choosing a path based on a condition.	if mark >= 50
Iteration	Repeating steps while a condition holds.	for number in range(1,6)
Syntax error	Violation of language grammar.	Missing colon
Runtime error	Error while the program runs.	Division by zero
Logical error	Program runs but gives a wrong result.	Wrong slab formula

CO1 • 9 hours • Applying

Module 1 — Computer, Operating System and Internet Applications

Identify hardware and interfaces, manage files and directories, and use Internet applications safely and effectively.

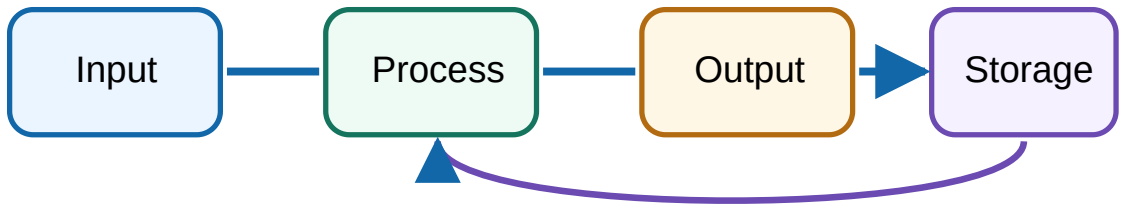
M1.01 • 2L + 2P

Computer functions, hardware, ports and cables

A computer accepts data and instructions, processes them, communicates results and stores data for later use.

Input–Process–Output–Storage cycle

Input supplies data or commands. The **CPU** processes them using instructions and working memory. **Output** communicates the result. **Storage** preserves files and programs.



Stored data can later return as input for another processing cycle.

CPU, memory and storage

The CPU repeatedly fetches, decodes and executes instructions. The ALU performs arithmetic and logic; the Control Unit coordinates activity; registers hold immediate values. RAM stores active programs temporarily. SSD/HDD stores files persistently.

> **Worked example:** When Calc adds five marks, the workbook and active values are in RAM, the CPU executes addition instructions, the total appears on the monitor and the saved workbook is written to SSD/HDD.

RAM power off □□□□□□□□ data □□□□□□□□□□□. SSD/HDD-□ save □□□□□ file power off □□□□□□□□ □□□□□□□□□□□.

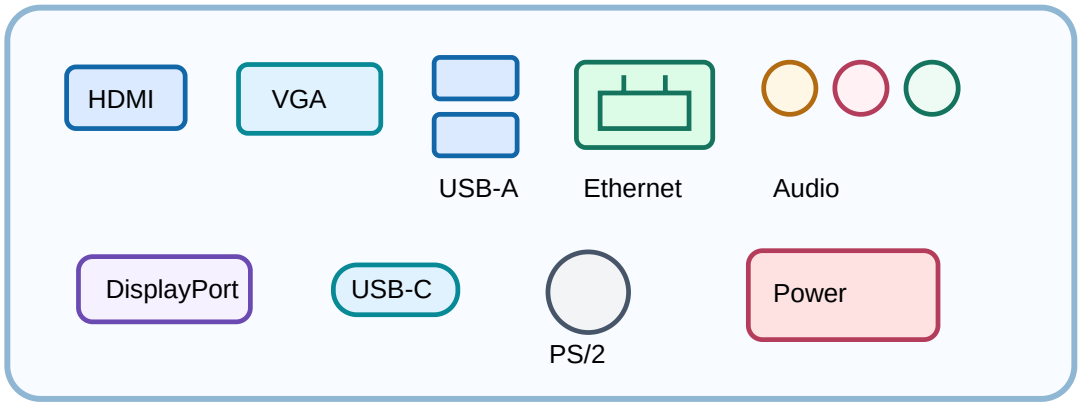
Hardware identification and troubleshooting

Component	Category	Purpose	Identifying feature	Basic troubleshooting clue
CPU	Processing	Executes instructions	Processor package in motherboard socket	Cooling or overheating problem
Motherboard	Connection/processing	Connects all major parts	Large main circuit board	Loose power/data cable or damaged slot
RAM	Primary memory	Temporary working storage	DIMM module	Module may not be fully seated
ROM/firmware chip	Non-volatile memory	Stores startup firmware	Small motherboard chip	Incorrect firmware update
HDD	Secondary storage	Magnetic long-term storage	2.5/3.5-inch drive	Cable problem or bad sectors
SSD	Secondary storage	Flash long-term storage	2.5-inch or M.2 device	Full drive or loose M.2 mounting
SMPS	Power	Converts AC to regulated DC	Metal power-supply enclosure	Fan or power-lead problem
Monitor	Output	Displays visual output	Screen with video inputs	Wrong input source or loose cable
Keyboard	Input	Enters text and commands	Key layout	USB/Bluetooth connection
Mouse	Input	Controls pointer	Buttons and optical sensor	Dirty sensor or weak battery
Scanner	Input	Digitises paper documents	Flatbed or document feeder	Driver or paper alignment
Printer	Output	Produces paper output	Inkjet/laser mechanism	Paper jam or empty cartridge
Webcam	Input	Captures video	Small camera/lens	Privacy permission blocked
Microphone	Input	Captures sound	Mic grille or plug	Wrong input selected
Speakers	Output	Produce sound	Speaker pair	Muted output or wrong port
Network adapter	Communication	Connects wired network	RJ45 socket	Cable/link or IP issue
Wi-Fi adapter	Communication	Connects wireless network	Internal/USB radio	Airplane mode or weak signal
Bluetooth adapter	Communication	Short-range wireless link	Internal/USB radio	Pairing mode disabled
UPS	Power protection	Temporary backup and surge protection	Battery-backed unit	Weak battery or overload
USB drive	Portable storage	Transfers files	USB connector	Unsafe removal or malware

Ports and interfaces

Interface	Appearance	Purpose	Cable/device	Caution	Status
USB Type-A	Flat rectangular	General data and power	Keyboard, mouse, flash drive	Insert in correct orientation	Current
USB Type-B	Nearly square	Printer/scanner connection	A-to-B cable	Do not confuse with Ethernet	Current/specialised
USB Type-C	Small reversible oval	Data, charging, sometimes video	USB-C cable	Capabilities vary by port/cable	Current
HDMI	Flat tapered digital connector	Digital video and audio	HDMI cable	Select correct monitor input	Current
VGA	15-pin D-sub	Analogue video	VGA cable	Avoid bent pins	Legacy
DisplayPort	Rectangle with clipped corner	Digital video/audio	DisplayPort cable	Release latch if present	Current
RJ45 Ethernet	Wide modular socket	Wired networking	Twisted-pair cable	Press latch before removal	Current
3.5 mm audio	Small round colour-coded jack	Headphone, speaker or microphone	3.5 mm plug	Use correct input/output jack	Current

Interface	Appearance	Purpose	Cable/device	Caution	Status
PS/2	Round six-pin socket	Legacy keyboard/mouse	PS/2 cable	Connect powered off	Legacy
Serial DE-9	Nine-pin D-sub	Industrial/legacy serial link	Serial cable	Verify pinout and settings	Legacy/industrial
Parallel DB-25	Twenty-five-pin D-sub	Legacy printer	Parallel cable	Tighten screws gently	Legacy
Power connector	Varies by device	Electrical power	Power cable/adapter	Match voltage and connector	Current
Memory-card slot	Thin card-sized slot	SD/microSD storage	Memory card	Correct adapter and direction	Current



Positions vary by computer. Match connector shape, label and intended device; never force a plug.

Practical 1 Identify and classify computer components

Aim

Identify external and demonstrated internal components.

Requirements

Computer, labelled images or supervised demonstration unit.

Expected result

Correct hardware inventory and classification.

Procedure

1. Shut down and disconnect power before an internal demonstration.
2. Observe each component without touching contacts.

3. Record name, category, feature, purpose and condition.
4. Ask the instructor to verify uncertain items.
5. Add labelled photographs only when permitted.

Observation table

Sl.No • component • category • identifying feature • purpose • condition.

> □□□□□□□□□□□□ □□□□□□: Component name English-□ □□□□□ □□□□□□; purpose □□□ clear sentence □□□ □□□□□.

Practical 2

Match ports and cables safely

1. Compare connector shape with the labelled diagram.
2. Check orientation and locking latch.
3. Insert gently without bending pins.
4. Record connected device and expected function.
5. Disconnect by holding the plug, not pulling the cable.

> **Precautions:** Never force a plug, use a damaged cable or remove a locking connector without releasing its latch.

Viva

1. Which interface carries digital audio and video?
2. What does RJ45 commonly connect?
3. Why are all USB-C ports not functionally identical?

M1.02 • 1L + 2P

Operating-system functions and file management

User interface

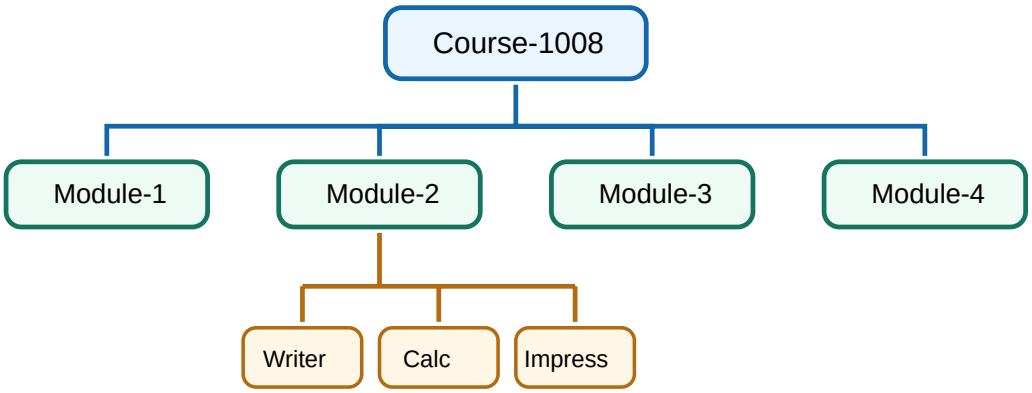
Desktop, application menu, windows, taskbar/dock, notifications, settings and accessibility features let the user control the system.

Resource management

The OS manages processes, memory, devices, files, users, permissions and networking.

File system

Files are organised in a hierarchy of drives/root, folders and subfolders. Paths identify exact locations.



A planned hierarchy prevents files from being mixed and simplifies backup.

Essential GUI operations

- ✓ Create folders and meaningful files.
- ✓ Copy, move, rename and search.
- ✓ Sort by name, type, size or modified date.
- ✓ View properties and file extensions.
- ✓ Compress and extract folders.
- ✓ Delete and restore from Recycle Bin/Trash.
- ✓ Create shortcuts without duplicating data.

Absolute and relative paths

An absolute path starts from the drive or root. A relative path starts from the current working directory.

```
Absolute Windows: C:\Course-1008\Module-4\Python\task1.py
Absolute Linux:   /home/student/Course-1008/Module-4/Python/task1.py
Relative:         Module-4/Python/task1.py
```

> **Common error:** A command can fail because the current folder is wrong, not because the file is missing.

Command reference

Action	Linux-style	Windows-style	Meaning
Current folder	`pwd`	`cd`	Displays current path.
List items	`ls`	`dir`	Lists files and folders.
Change folder	`cd Module-1`	`cd Module-1`	Moves into a directory.
Create folder	`mkdir Python`	`mkdir Python` or `md Python`	Creates a directory.

Action	Linux-style	Windows-style	Meaning
Remove empty folder	`rmdir Old`	`rmdir Old` or `rd Old`	Removes an empty directory.
Copy file	`cp notes.txt backup.txt`	`copy notes.txt backup.txt`	Creates a copy.
Move file	`mv shot.png Module-1/`	`move shot.png Module-1\`	Moves an item.
Rename	`mv old.txt new.txt`	`ren old.txt new.txt`	Changes the name.
Delete file	`rm draft.txt`	`del draft.txt`	Deletes after path verification.
Show text	`cat notes.txt`	`type notes.txt`	Displays text-file content.
Clear screen	`clear`	`cls`	Clears terminal display.

```
Required practical result:
Course-1008/
├── Module-1/
├── Module-2/
│   ├── Writer/
│   ├── Calc/
│   └── Impress/
├── Module-3/
├── Module-4/
│   └── Python/
└── Projects/
```

Practical 3 Create, organise, search and restore files

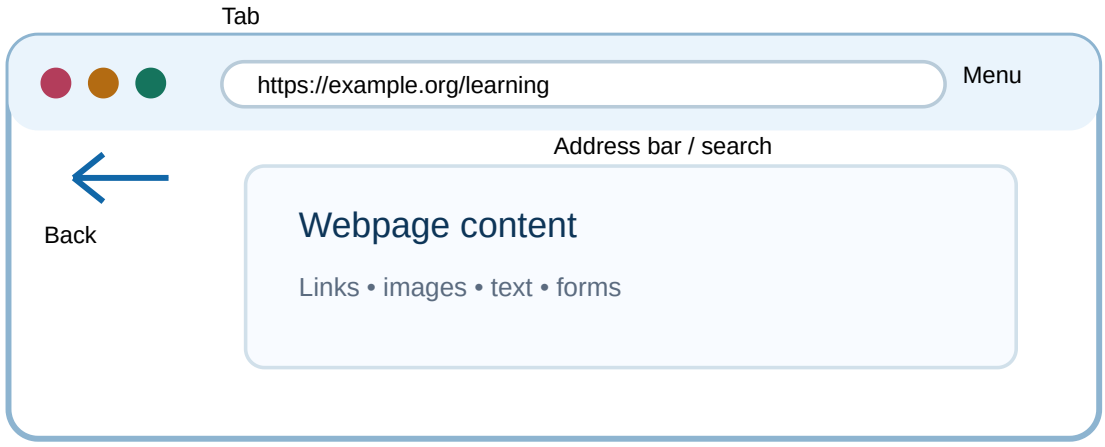
1. Create the complete folder hierarchy.
2. Create sample text and Python files.
3. Rename one draft using a version number.
4. Copy one file to Backup and move a screenshot to the correct module.
5. Search by partial name and extension.
6. Delete a test file and restore it.
7. Capture the final hierarchy and command history.

> **Delete safely:** Check current path and target name. Command-line deletion may bypass Recycle Bin/Trash.

Command □□□□□□□□□□□□□□□□□ □□□□□□ current folder verify □□□□□□□; wrong path data loss □□□□□□□□□□.

M1.03 • 2P

Browser, search engine, email and web portals



A browser opens URLs, manages tabs and downloads, stores history and controls permissions.

Search effectively

- Use specific keywords.
- Use quotation marks for an exact phrase.
- Use `site:` for one website.
- Use `filetype:pdf` for PDFs.
- Exclude an unwanted term where supported.

Evaluate a result

- Who is the author or organisation?
- When was it published or updated?
- Does it cite evidence?
- Is the domain appropriate?
- Can the claim be confirmed elsewhere?

Browser safety

- Check the complete domain.
- Prefer HTTPS for sensitive actions.
- Review download and permission prompts.
- Do not assume the first result is best.
- Private mode does not hide activity from every service or network.

Search exercises

1. Search the exact phrase “Introduction to IT Systems Lab”.

2. Find Python material only on the official Python website using `site:python.org`.
3. Find a PDF using `filetype:pdf`.
4. Compare two sources for the same technical definition.
5. Identify author, date and evidence on a webpage.
6. Bookmark a useful educational page.
7. Download a permitted PDF and organise it.
8. Inspect site permissions under supervision.

Professional email model

To: faculty@example.edu

Subject: Course 1008 – Project Report – Team 3

Dear Sir/Madam,

Please find attached our Course 1008 project report titled “Digital Safety for Students”. The report has been checked and exported as PDF.

Regards,
Team 3
Semester 1

Attachment: 1008_ProjectReport_Team03.pdf

Mistake	Why it matters	Correction
Empty/vague subject	Purpose is unclear.	Use course, task and team/name.
Forgotten attachment	Required work is missing.	Attach, verify filename and preview.
Reply All unnecessarily	Sends to everyone.	Choose Reply or Reply All intentionally.
All recipients visible	May expose addresses.	Use BCC when privacy is required.
Suspicious attachment opened	May contain malware.	Verify sender and context; report it.

Practical 4 Compose a professional email with an attachment

1. Use the correct recipient supplied by the instructor.
2. Write a meaningful subject.
3. Add greeting, concise message and signature.

4. Attach the correct PDF and wait for upload.
5. Verify filename, size and preview.
6. Proofread recipient, subject and body.
7. Log out on a shared computer.

□□□□□□□□: Send □□□□□□□□□□□□□□□ □□□□□□□ recipient, subject, attachment □□□□□□□ □□□□□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□□□.

Web portals: Educational, examination, government-service, cloud-storage and learning portals provide different services. Use demonstration data and never publish credentials in a record.

CO2 • 11 hours • Applying

Module 2 — Stand-alone and Cloud Office Tools

Create professional documents, spreadsheets and presentations, then collaborate, verify and export safely.

M2.01 • 1L

OpenOffice suite and file formats

OpenOffice is an office-productivity suite. Its applications share common menus, toolbars, file operations and export facilities, while each application is specialised for a particular type of work.

Component	Main purpose	Native format	Example activity
Writer	Word processing	.odt	Project report or formal letter
Calc	Spreadsheet and calculation	.ods	Rank list, electricity bill or payroll
Impress	Presentation	.odp	Course presentation
Draw	Vector drawing and diagrams	.odg	Simple technical diagram
Base	Database front-end	.odb	Record database
Math	Formula editor	.odf	Mathematical expression

Common interface elements

Title bar, menu bar, toolbars, document area, sidebar, navigator and status bar. **Save** updates the current file. **Save As** creates another name, location or format. **Export** creates an output such as PDF.

Compatibility and verification

OpenDocument formats preserve OpenOffice features. Microsoft formats may be needed for exchange, but complex formatting can change. Keep the editable original and inspect the exported file.

Editable original file (.odt/.ods/.odp) □□□□□□□□□□□□; PDF/export □□□□□□□ verify □□□□□ □□□□ submit □□□□□□□.

> **Practical rule:** Save early, use meaningful filenames, create versioned backups, and never assume that an exported file looks exactly like the editable source until it is opened and checked.

M2.02 • 3P

OpenOffice Writer — professional document preparation

Page setup
Size, orientation, margins and

Paragraph
Alignment, spacing and

Styles
Consistent headings and body

page style.

indentation.

text.

Objects
Tables, images, captions and wrapping.

Long document
Header, footer, page number and TOC.

Output
Spell check, print preview and PDF export.

Character formatting

Font family, size, bold, italic, underline, colour and highlighting affect selected characters. Use emphasis sparingly and consistently.

Paragraph formatting

Alignment, line spacing, paragraph spacing, indentation, bullets and numbering affect complete paragraphs.

Styles

Styles store reusable formatting rules. Heading styles make navigation and automatic table-of-contents generation possible.

Solved practical A — formal letter

1. Create a new Writer document and set A4 portrait with suitable margins.
2. Type sender address and date using consistent alignment.
3. Add recipient address, subject and salutation.
4. Write short paragraphs with readable spacing.
5. Add closing, name and designation.
6. Run spelling check and inspect print preview.
7. Save as ODT and export a verified PDF.

Student Name
Semester 1
Government Polytechnic
16 June 2026

To
The Faculty in Charge
Introduction to IT Systems Lab

Subject: Submission of Course 1008 Practical Record

Respected Sir/Madam,

I am submitting my completed Course 1008 practical record for verification. The document contains the required activities, screenshots, results and corrections.

Yours faithfully,
Student Name
Roll No.

> **Formatting specification:** Body 11–12 pt, one readable font family, left alignment for paragraphs, bold subject, no decorative effects, and consistent paragraph spacing.

Solved practical B — ten-page project report

Section	Required content	Writer feature
Cover page	Title, course, team, institution and year	Page style and centred layout
Certificate	Prescribed certification text and signature space	Paragraph/table layout
Acknowledgement	Brief formal acknowledgement	Body style
Table of contents	Automatic headings and page numbers	Heading styles + TOC
Chapters	Introduction, objectives, work, observations and conclusion	Heading 1/2 and page breaks
Images and tables	Relevant labelled evidence	Insert, wrap and captions
Header/footer	Course/project and page number	Page style fields
References	Sources used	Consistent list

Exact workflow

1. Plan chapters and create a project folder.
2. Configure page style before heavy formatting.
3. Apply Heading styles instead of manually enlarging every heading.
4. Insert page breaks between front matter and chapters.
5. Insert images, resize proportionally and add captions.
6. Create tables with clear headers.
7. Insert header/footer and page-number field.
8. Generate and update the TOC.
9. Check spelling, layout and references.
10. Save editable ODT, export PDF and verify every page.

Common mistakes and corrections

- **Repeated spaces for alignment:** use tabs, paragraph settings or tables.
- **Manual heading formatting:** apply styles.
- **Distorted image:** preserve aspect ratio.
- **Missing caption/source:** label and acknowledge.
- **Typed page number:** insert a page-number field.
- **Outdated TOC:** update it after editing.
- **Only PDF saved:** retain editable source when required.

> TOC automatic □□□□□□ headings-□ proper Styles apply □□□□□□□. Manual bold/size □□□□□□ □□□□□□□.

Writer record task

Prepare and submit a structured report

Aim

Create a styled report with images, table, TOC, header/footer and PDF export.

Evidence

ODT source, exported PDF and screenshots of styles/TOC.

Result

A consistent, navigable and print-ready document.

Record checkpoints

- ✓ Correct filename and folder
- ✓ Heading styles visible in Navigator
- ✓ Captions and table headers present
- ✓ TOC updated
- ✓ Page numbers correct
- ✓ PDF opened and checked

M2.03 • 2P

OpenOffice Calc — spreadsheets, formulas and solved applications

Cell and range

A cell is addressed by column and row, such as B4. A range such as B4:F4 contains multiple cells.

Formula and function

A formula begins with =. A function such as SUM performs a predefined calculation.

References

Relative references change when copied. Absolute references such as $B$2 remain fixed.

Function/operator	Example	Meaning
Add/subtract/multiply/divide	`=B2+C2-D2`, `=B2*C2`, `=B2/C2`	Arithmetic
SUM	`=SUM(C2:G2)`	Total of a range
AVERAGE	`=AVERAGE(C2:G2)`	Mean
MIN / MAX	`=MIN(C2:G2)`	Smallest/largest
COUNT	`=COUNT(C2:G2)`	Numeric cell count
IF	`=IF(H2>=50;"PASS";"FAIL")`	Conditional result
Absolute reference	`=B2*$F$1`	Locks common rate cell

Fully solved spreadsheet tasks

Practical	Suggested layout	Core formulas	Verification
Mark sheet	Roll, name, five marks, total, average, percentage, result and rank	`=SUM(C2:G2)`; `=AVERAGE(C2:G2)`; `=H2/500*100`; `=IF(MIN(C2:G2)>=40;"PASS";"FAIL")`	Calculate one student manually and check failed-subject logic.
Rank list	Student identity, total/percentage and rank	`=RANK(H2;$H$2:$H$31;0)` or supported equivalent	Sort rank ascending and inspect duplicate totals.
Monthly expense	Date, category, description and amount	`=SUM(D2:D31)` and category summaries	Format currency and compare chart with table.
Electricity bill	Consumer, units, slab charge, fixed charge and total	`=IF(B2<=100;B2*1.5;IF(B2<=200;150+(B2-100)*2.5;400+(B2-200)*4))`	Test 0, 100, 101, 200 and 201 units.
Payroll	Employee, basic, HRA, DA, deduction, gross and net	`=B2*20%`; `=B2*10%`; gross and net formulas	Use absolute references for common rates stored separately.
Chart	Label column plus numeric series	Insert → Chart; choose a suitable type	Check selected range, title, legend and axes.

> **Slab warning:** Do not multiply every unit by the highest reached rate. Calculate each slab separately and test every boundary.

> **Calc tip:** Formula type □□□□□ □□□□ one row manually verify □□□□□□□. Fill handle □□□□□□□□□□□□□□□□□ □□□□□□ relative/absolute reference □□□□□□□ □□□□□□□□□□□□.

Calc record task — Create a mark sheet with result and rank

1. Enter headings and five demonstration records.
2. Apply number formatting and borders.
3. Enter total, average, percentage and pass/fail formulas.

4. Copy formulas with the fill handle.
5. Add rank using an absolute range.
6. Sort a copy by rank.
7. Create a chart using names and totals.
8. Verify at least one row manually and save the workbook.

> **Expected evidence:** Formula bar screenshot, completed table, sorted rank list and chart.

M2.04 • 2P

OpenOffice Impress — presentation design and delivery

Presentation workflow

1. Define audience and purpose.
2. Create a slide outline.
3. Choose a consistent theme or master slide.
4. Use one main idea per slide.
5. Add relevant images, tables or charts.
6. Use transitions and animations sparingly.
7. Add speaker notes and rehearse timing.
8. Test audio/video on the presentation computer.
9. Export and keep a backup.

Required course presentation

1. Title
2. Course overview
3. Computer components
4. Operating system
5. Internet and email

6. Writer
7. Calc
8. Impress
9. Algorithm and flowchart
10. Python
11. Summary
12. References

> **Design rules:** Large readable text, strong contrast, limited bullet points, relevant visuals, consistent alignment, source acknowledgement and no distracting animation.

Embedded media practical

Create a short presentation containing one permitted audio clip and one permitted video clip. Insert media, configure playback, test the file path or embedding, copy the project folder to another computer and verify that the presentation still works.

Problem	Cause	Correction
Text too small	Too much content on one slide	Split the content and enlarge text
Media does not play	Unsupported format or broken link	Use supported format and test on target system
Different appearance	Missing font/theme or compatibility issue	Use common fonts, export backup PDF and verify
Animation distracts	Too many effects	Use only effects that support meaning

M2.05 • 3P

Cloud documents, spreadsheets and presentations

Area	Stand-alone office	Cloud office
Internet	Usually not required for basic editing	Normally required for full collaboration
Storage	Local drive	Remote account storage
Collaboration	Files exchanged manually	Simultaneous editing and comments
Version history	Manual versions unless app supports history	Often built in
Privacy	Controlled by local access and backup	Controlled by account, sharing and provider policy
Availability	Installed computer	Authorised devices
Compatibility	Depends on installed software	Browser-based; export may change formatting

Cloud collaboration practical

1. Create a project folder in cloud storage.
2. Upload or create a document, spreadsheet and presentation.
3. Share the document with comment-only permission.
4. Share the spreadsheet with edit permission only to the team.

5. Add a comment and resolve it after correction.
6. Inspect version history and identify an earlier edit.
7. Download/export each file in a suitable format.
8. Remove unnecessary sharing and log out from the shared computer.

Viewer

Can read but normally cannot comment or edit.

Commenter

Can add feedback without directly changing content.

Editor

Can change content; grant only when required.

> **Privacy:** “Anyone with the link” may be inappropriate. Use restricted sharing and the least permission needed.

SERIES TEST I • 1 HOUR

Original practice test — Modules 1 and 2

This is a syllabus-based practice test, not an official board question paper.

1. Name one volatile and one non-volatile memory.
2. Differentiate copy and move.
3. State one use each of HDMI and RJ45.
4. Write two signs of a phishing email.
5. Explain Save, Save As and Export.
6. Create the prescribed Course-1008 folder structure and show evidence.
7. Prepare a one-page formal letter in Writer and export it as PDF.
8. In Calc, create a five-student mark sheet with total, average and result.
9. Plan five slides explaining safe Internet use.
10. Explain Viewer, Commenter and Editor permissions.

CO3 • M3.01 • 5L • Applying

Module 3 — Algorithms and Flowcharts

Analyse a problem, state input and output, design a finite solution and trace sequence, selection and iteration before writing code.

Problem-solving foundation

A good solution begins before programming. Clarify the task, inputs, outputs, restrictions and test cases, then write and verify an algorithm.

Problem definition

State what must be calculated or decided, available inputs, expected output and constraints.

Algorithm

An ordered, finite and unambiguous sequence of effective steps that produces the required result.

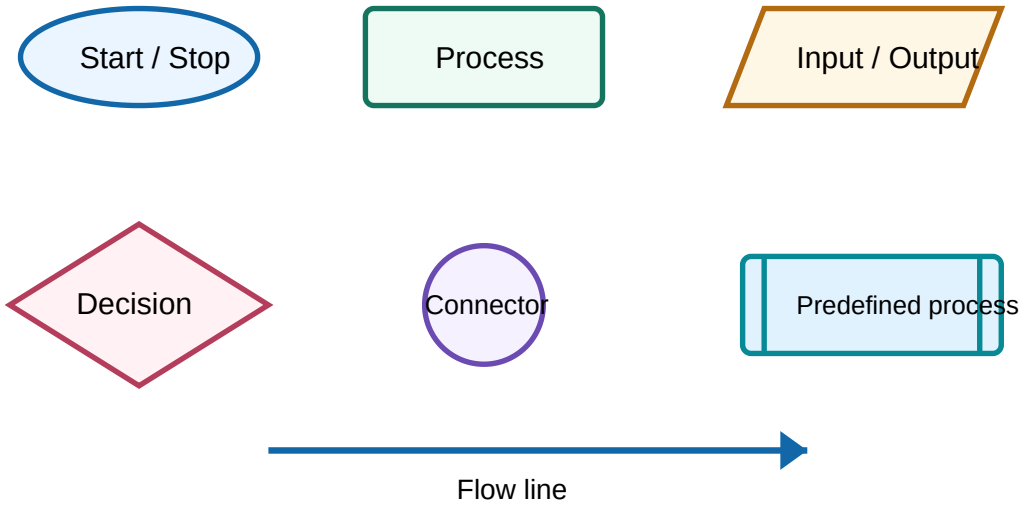
Dry run

Execute the steps manually with test data and record variable changes in a trace table.

Characteristic	Meaning	Check question
Input	Zero or more clearly identified values	What values must be supplied?
Output	At least one meaningful result	What should be displayed or stored?
Definiteness	Every step is precise and unambiguous	Could two readers interpret the step differently?
Finiteness	Terminates after a finite number of steps	What makes repetition stop?
Effectiveness	Each step is practical and executable	Can a person or computer perform it?

□□□□□□□□□: Problem □□□□□□□□, input □□□□□□□□, output □□□□□□□□, decision/loop □□□□□□□□□□ □□□□□□□ □□□□□ □□□□□□□. □□□□ □□□□□□□□□□ flowchart □□□□□□□□□□□ code.

Flowchart symbols and rules



Decision branches must be labelled, commonly Yes/No or True/False.

Drawing rules

- ✓ Use one Start and at least one Stop.
- ✓ Use the correct symbol for each action.
- ✓ Draw flow from top to bottom or left to right.
- ✓ Label decision branches.
- ✓ Use connectors rather than long crossed lines.
- ✓ Show initialisation and update in every loop.

Frequent mistakes

- Missing Start/Stop.
- Decision with only one outgoing branch.
- Input written inside a process rectangle.
- Arrow returning to the wrong step.
- Counter never updated.
- Condition tested after the wrong operation.
- Using an unclear variable name.

SEQUENCE

Solved sequence problems

Sequence executes each instruction once in the written order.

1. Add two numbers

Input: a, b
Output: sum

```
START
READ a, b
sum ← a + b
PRINT sum
STOP
```

Dry run:

a = 12, b = 8 → sum = 20.

Practice:

Modify to calculate sum, difference and product.

2. Rectangle area and perimeter

```
START
READ length, breadth
area ← length × breadth
perimeter ← 2 × (length + breadth)
PRINT area, perimeter
STOP
```

For length 5 and breadth 3: area = 15; perimeter = 16.

> **Common error:** Writing `2 × length + breadth` without brackets gives a different expression.

3. Celsius to Fahrenheit

```
START
READ celsius
fahrenheit ← celsius × 9 / 5 + 32
PRINT fahrenheit
STOP
```

25°C → 25 × 9 / 5 + 32 = 77°F.

Test 0°C, 100°C and a negative temperature.

4. Simple interest

```
START
READ principal, rate, time
interest ← principal × rate × time / 100
PRINT interest
STOP
```

P = 10000, R = 8, T = 2 → interest = 1600.

Add amount = principal + interest as a variation.

5. Total and average of three marks

```
START
READ m1, m2, m3
total ← m1 + m2 + m3
average ← total / 3
PRINT total, average
STOP
```

70, 80, 90 → total 240, average 80.

Validate marks between 0 and 100 as a later selection exercise.

6. Exchange two values using a temporary variable

```
START
READ a, b
temporary ← a
a ← b
b ← temporary
PRINT a, b
STOP
```

a = 5, b = 9 → after exchange a = 9, b = 5.

The temporary variable prevents the original value of a from being lost.

SELECTION

Solved decision problems

Selection chooses a path according to a Boolean condition.

7. Even or odd

```
START
READ number
IF number mod 2 = 0 THEN
    PRINT "Even"
ELSE
    PRINT "Odd"
END IF
STOP
```

17 mod 2 = 1, so the result is Odd.

Zero is even because its remainder on division by 2 is zero.

8. Positive, negative or zero

```
START
READ number
IF number > 0 THEN
    PRINT "Positive"
ELSE IF number < 0 THEN
    PRINT "Negative"
ELSE
    PRINT "Zero"
END IF
STOP
```

Use three test cases: -4, 0 and 7.

The final else represents exactly zero after the two earlier conditions are false.

9. Largest of three values

```
START
READ a, b, c
largest ← a
```

```
IF b > largest THEN largest ← b
IF c > largest THEN largest ← c
PRINT largest
STOP
```

For 4, 9, 7: largest begins at 4, becomes 9 and remains 9.

This method also handles equal values.

10. Pass/fail and grade classification

```
START
READ mark
IF mark < 0 OR mark > 100 THEN
    PRINT "Invalid"
ELSE IF mark >= 90 THEN PRINT "A+"
ELSE IF mark >= 80 THEN PRINT "A"
ELSE IF mark >= 70 THEN PRINT "B"
ELSE IF mark >= 60 THEN PRINT "C"
ELSE IF mark >= 50 THEN PRINT "D"
ELSE PRINT "F"
STOP
```

Check highest boundary first. If mark 94 is checked against 50 first, the more specific grade would never be reached.

Boundary tests: -1, 0, 49, 50, 89, 90, 100 and 101.

11. Leap-year decision

```
START
READ year
IF year mod 400 = 0 OR
   (year mod 4 = 0 AND year mod 100 ≠ 0) THEN
    PRINT "Leap year"
ELSE
    PRINT "Not leap year"
END IF
STOP
```

2000 is leap; 1900 is not; 2024 is leap; 2023 is not.

The century rule prevents every multiple of 4 from being accepted.

12. Electricity-bill slab decision

```
START
READ units
IF units < 0 THEN PRINT "Invalid"
ELSE IF units <= 100 THEN
    charge ← units × 1.50
ELSE IF units <= 200 THEN
    charge ← 100 × 1.50 + (units - 100) × 2.50
ELSE
    charge ← 100 × 1.50 + 100 × 2.50
              + (units - 200) × 4.00
PRINT charge
STOP
```

For 250 units: 150 + 250 + 200 = 600.

> **Important:** Do not multiply all 250 units by ₹4. Each slab is charged separately.

ITERATION

Solved loop problems

13. Print 1 to n and find the sum

```
START
READ n
total ← 0
FOR number ← 1 TO n
    PRINT number
    total ← total + number
END FOR
PRINT total
STOP
```

n = 5: total changes 0 → 1 → 3 → 6 → 10 → 15.

Counter: number. **Accumulator:** total.

14. Multiplication table

```
START
READ number
FOR multiplier ← 1 TO 10
    product ← number × multiplier
    PRINT number, multiplier, product
END FOR
STOP
```

For number 7, the loop prints 7×1 through 7×10.

Initialisation, condition and update must all be represented.

15. Factorial

```
START
READ n
IF n < 0 THEN PRINT "Invalid"
ELSE
    factorial ← 1
    FOR value ← 2 TO n
        factorial ← factorial × value
    END FOR
    PRINT factorial
END IF
STOP
```

5! = 1×2×3×4×5 = 120. The accumulator begins at 1 because zero would make every product zero.

0! is 1; the loop can run zero times.

16. Reverse a number and test palindrome

```
START
READ number
```

```
copy ← number
reverse ← 0
WHILE copy > 0
    digit ← copy mod 10
    reverse ← reverse × 10 + digit
    copy ← copy div 10
END WHILE
IF reverse = number THEN PRINT "Palindrome"
ELSE PRINT "Not palindrome"
STOP
```

For 121: reverse changes 0 → 1 → 12 → 121, so it is a palindrome.

Keep the original number because the loop changes copy.

17. Prime-number test

```
START
READ n
prime ← n >= 2
divisor ← 2
WHILE divisor × divisor <= n AND prime
    IF n mod divisor = 0 THEN prime ← false
    divisor ← divisor + 1
END WHILE
PRINT result
STOP
```

For 29, test divisors 2, 3, 4 and 5. No divisor divides exactly, so 29 is prime.

Testing up to the square root is sufficient because factors occur in pairs.

18. Fibonacci sequence

```
START
READ terms
a ← 0
b ← 1
REPEAT terms TIMES
    PRINT a
    next ← a + b
    a ← b
    b ← next
END REPEAT
STOP
```

Six terms: 0, 1, 1, 2, 3, 5.

Two state variables are updated each iteration.

Comparison and conversion to Python

Concept	Use	Pseudocode	Python form
Sequence	One ordered path	READ → calculate → PRINT	Normal statements
Selection	Choose a path	IF / ELSE	`if`, `elif`, `else`
Iteration	Repeat steps	FOR / WHILE	`for`, `while`
Input/output symbol	Read/display	READ n / PRINT result	`input()` / `print()`

Concept	Use	Pseudocode	Python form
Process symbol	Assignment/calculation	total ← a + b	`total = a + b`

> **Exam/lab writing order:** Problem → Input/Output → Algorithm → Flowchart → Dry run → Result. The Python program in Module 4 must follow the same logic.

CO4 • 18 hours • Applying

Module 4 — Python Programming

Translate algorithms into Python 3 programs using interactive input/output, arithmetic expressions, selection and iteration.

M4.01 • 1L + 2P

Python environment, input, output and arithmetic expressions

Interactive mode

Commands execute immediately at the prompt. It is useful for testing expressions and small statements.

Script mode

Statements are saved in a `.py` file and executed as a complete program.

Indentation

Python uses indentation to define blocks. Inconsistent indentation causes an error or incorrect structure.

Item	Explanation	Example
Comment	Text ignored by Python; explains code.	`# calculate total`
Variable	Name referring to a value.	`total = 50`
int	Whole number.	`25`
float	Number with fractional part.	`12.5`
str	Text sequence.	`"Kerala"`
bool	Truth value.	`True`, `False`
input()	Reads text from the user.	`name = input("Name: ")`
print()	Displays values.	`print(total)`
Conversion	Changes input text to a numeric type.	`age = int(input("Age: "))`
f-string	Embeds values in formatted text.	`print(f"Total = {total}")`

Operator	Meaning	Example	Result
+	Addition	`5 + 2`	7
-	Subtraction	`5 - 2`	3
*	Multiplication	`5 * 2`	10
/	True division	`5 / 2`	2.5
//	Floor division	`5 // 2`	2
%	Remainder	`5 % 2`	1
**	Power	`2 ** 3`	8

> **Input conversion:** `input()` returns text. Without `int()` or `float()`, `"2" + "3"` gives `"23"`, not 5.

M4.02 • 2L + 4P

Selection statements, relational and logical operators

Group	Operators	Example
Relational	`== != < <= > >=`	`mark >= 50`
Logical AND	`and`	Both conditions must be true.
Logical OR	`or`	At least one condition is true.
Logical NOT	`not`	Reverses the truth value.

A	B	A and B	A or B	not A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

Simple if

Executes a block only when the condition is true.

if-else

Selects exactly one of two paths.

Nested if / elif

Handles multiple related decisions, ranges and priorities.

> **Boundary testing:** For marks, units, age or salary, test values just below, equal to and just above every boundary.

M4.03 • 1L + 2P

Iteration with for and while

Construct	Best use	Critical point
`for`	Known sequence or count-controlled repetition	`range` stop is excluded.
`while`	Repeat while a condition is true	Initialise and update state.
Counter	Counts events or iterations	Usually increases by one.
Accumulator	Builds sum, product or result	Choose correct initial value.
`break`	Leaves nearest loop	Use intentionally.

Construct	Best use	Critical point
`continue`	Skips to next iteration	Do not skip a required update.

Thirty-four solved Python programs

Every program is complete Python 3. Copy it, run it, change input and compare the output with a manual dry run.

Program 1 Welcome message

```
print("Welcome to Introduction to IT Systems Lab")
```

Output: Welcome to Introduction to IT Systems Lab

Practice: Display course code and semester on separate lines.

Program 2 Read and display student details

```
name = input("Name: ")
roll = input("Roll number: ")
print(f"{roll} - {name}")
```

Logic: Input remains text and an f-string formats the output.

> Roll number arithmetic □□□□□□□□□□□□□□□□□□□□ string □□□ □□□□□□□□□□□.

Program 3 Add two numbers

```
a = float(input("First number: "))
b = float(input("Second number: "))
print(f"Sum = {a + b}")
```

Sample: 12.5 and 3 → Sum = 15.5.

Program 4 All arithmetic operations

```
a = float(input("a: "))
b = float(input("b: "))
print("Add:", a + b)
print("Subtract:", a - b)
print("Multiply:", a * b)
if b != 0:
    print("Divide:", a / b)
else:
    print("Division undefined")
```

Common mistake: Division must be protected when b is zero.

Program 5 Rectangle area and perimeter

```
length = float(input("Length: "))
breadth = float(input("Breadth: "))
print("Area =", length * breadth)
print("Perimeter =", 2 * (length + breadth))
```

Dry run: 5 and 3 → area 15, perimeter 16.

Program 6 Circle area and circumference

```
import math
radius = float(input("Radius: "))
print(f"Area = {math.pi * radius ** 2:.2f}")
print(f"Circumference = {2 * math.pi * radius:.2f}")
```

Practice: Reject a negative radius.

Program 7 Celsius to Fahrenheit

```
celsius = float(input("Celsius: "))
fahrenheit = celsius * 9 / 5 + 32
print(f"Fahrenheit = {fahrenheit:.2f}")
```

Test: 0 → 32; 100 → 212.

Program 8 Simple interest

```
principal = float(input("Principal: "))
rate = float(input("Rate %: "))
time = float(input("Time in years: "))
interest = principal * rate * time / 100
print(f"Simple interest = {interest:.2f}")
```

Dry run: 10000, 8, 2 → 1600.

Program 9 Total, average and percentage

```
m1 = float(input("Mark 1: "))
m2 = float(input("Mark 2: "))
m3 = float(input("Mark 3: "))
total = m1 + m2 + m3
print("Total =", total)
print("Average =", total / 3)
print("Percentage =", total / 300 * 100)
```

Practice: Validate every mark between 0 and 100.

Program 10 Swap two variables

```
a = input("a: ")
b = input("b: ")
a, b = b, a
print("a =", a, "b =", b)
```

Python supports tuple unpacking, so a temporary variable is not required here.

Program 11 Convert total seconds

```
seconds = int(input("Total seconds: "))
hours = seconds // 3600
remaining = seconds % 3600
minutes = remaining // 60
seconds_left = remaining % 60
print(hours, "h", minutes, "m", seconds_left, "s")
```

Sample: 3672 → 1 h 1 m 12 s.

Program 12 Gross salary

```
basic = float(input("Basic pay: "))
hra = basic * 0.20
da = basic * 0.10
gross = basic + hra + da
print(f"Gross salary = {gross:.2f}")
```

Practice: Add a deduction and calculate net salary.

Program 13 Positive, negative or zero

```
number = float(input("Number: "))
if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Test -4, 0 and 7.

Program 14 Even or odd

```
number = int(input("Integer: "))
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Zero is even because `0 % 2` is zero.

Program 15 Largest of three

```
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))
largest = a
if b > largest:
    largest = b
if c > largest:
    largest = c
print("Largest =", largest)
```

This method handles equal values without complex nesting.

Program 16 Grade with input validation

```
mark = float(input("Mark: "))
if not 0 <= mark <= 100:
    print("Invalid mark")
elif mark >= 90:
    print("A+")
elif mark >= 80:
    print("A")
elif mark >= 70:
    print("B")
elif mark >= 60:
    print("C")
elif mark >= 50:
    print("D")
else:
    print("F")
```

Boundary tests: -1, 0, 49, 50, 89, 90, 100 and 101.

Program 17 Voting eligibility

```
age = int(input("Age: "))
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible")
```

Practice: Reject negative age.

Program 18 Leap year

```
year = int(input("Year: "))
if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):
    print("Leap year")
else:
    print("Not a leap year")
```

Test 1900, 2000, 2023 and 2024.

Program 19 Divisible by 3 and 5

```
number = int(input("Integer: "))
if number % 3 == 0 and number % 5 == 0:
    print("Divisible by both")
else:
    print("Not divisible by both")
```

Both conditions must be true.

Program 20 Simple calculator

```
a = float(input("First: "))
op = input("Operator (+ - * /): ")
b = float(input("Second: "))
if op == "+":
    print(a + b)
elif op == "-":
```

```
    print(a - b)
elif op == "*":
    print(a * b)
elif op == "/" and b != 0:
    print(a / b)
else:
    print("Invalid operation")
```

Validate both the operator and the zero-divisor case.

Program 21 Electricity bill using slabs

```
units = int(input("Units: "))
if units < 0:
    print("Invalid units")
elif units <= 100:
    charge = units * 1.50
elif units <= 200:
    charge = 100 * 1.50 + (units - 100) * 2.50
else:
    charge = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00
if units >= 0:
    print(f"Charge = {charge:.2f}")
```

For 250 units: 150 + 250 + 200 = 600.

> Highest rate □□□□□□ units-□□□ apply □□□□□□□□□. □□□ slab □□□□□□□□□□□ calculate □□□□□□□.

Program 22 Triangle validity and type

```
a = float(input("Side a: "))
b = float(input("Side b: "))
c = float(input("Side c: "))
if a <= 0 or b <= 0 or c <= 0 or a+b <= c or a+c <= b or b+c <= a:
    print("Invalid triangle")
elif a == b == c:
    print("Equilateral")
elif a == b or b == c or a == c:
    print("Isosceles")
else:
    print("Scalene")
```

Validity must be checked before classification.

Program 23 Print 1 to n

```
n = int(input("n: "))
for number in range(1, n + 1):
    print(number, end=" ")
```

`range` excludes its stop value, so use `n + 1`.

Program 24 Sum 1 to n

```
n = int(input("n: "))
total = 0
for number in range(1, n + 1):
```

```
    total += number
print("Sum =", total)
```

Trace for 5: 0 → 1 → 3 → 6 → 10 → 15.

Program 25 Multiplication table

```
number = int(input("Number: "))
for multiplier in range(1, 11):
    print(f"{number} x {multiplier} = {number * multiplier}")
```

Count-controlled repetition from 1 to 10.

Program 26 Factorial

```
n = int(input("n: "))
if n < 0:
    print("Undefined for negative integers")
else:
    factorial = 1
    for value in range(2, n + 1):
        factorial *= value
    print("Factorial =", factorial)
```

The product accumulator must begin at 1.

Program 27 Sum of digits

```
number = abs(int(input("Integer: ")))
total = 0
while number > 0:
    total += number % 10
    number //= 10
print("Digit sum =", total)
```

Trace 572: 2 + 7 + 5 = 14.

Program 28 Reverse a number

```
number = abs(int(input("Integer: ")))
reverse = 0
while number > 0:
    reverse = reverse * 10 + number % 10
    number //= 10
print("Reverse =", reverse)
```

Each extracted digit is appended to the right of reverse.

Program 29 Palindrome number

```
number = int(input("Positive integer: "))
copy = number
reverse = 0
while copy > 0:
    reverse = reverse * 10 + copy % 10
    copy //= 10
print("Palindrome" if number == reverse else "Not palindrome")
```

Keep the original number for the final comparison.

Program 30 Armstrong number

```
number = int(input("Non-negative integer: "))
digits = str(number)
power = len(digits)
total = sum(int(digit) ** power for digit in digits)
print("Armstrong" if total == number else "Not Armstrong")
```

153 = $1^3 + 5^3 + 3^3$.

Program 31 Prime-number test

```
number = int(input("Integer: "))
prime = number >= 2
divisor = 2
while divisor * divisor <= number and prime:
    if number % divisor == 0:
        prime = False
    divisor += 1
print("Prime" if prime else "Not prime")
```

Testing up to the square root is sufficient.

Program 32 Fibonacci series

```
terms = int(input("Number of terms: "))
a, b = 0, 1
for _ in range(max(0, terms)):
    print(a, end=" ")
    a, b = b, a + b
```

Six terms: 0 1 1 2 3 5.

Program 33 Greatest common divisor

```
a = abs(int(input("a: ")))
b = abs(int(input("b: ")))
while b != 0:
    a, b = b, a % b
print("GCD =", a)
```

Euclid's algorithm repeatedly replaces the pair with divisor and remainder.

Program 34 Star triangle

```
rows = int(input("Rows: "))
for row in range(1, rows + 1):
    print("*" * row)
```

Output for 4:

```
*
**
***
****
```

SERIES TEST II • 1 HOUR

Original practice test — Algorithms and Python

1. Define algorithm and state three characteristics.
2. Draw and name four flowchart symbols.
3. Write an algorithm to find the larger of two numbers.
4. Dry-run the sum-of-digits algorithm for 572.
5. Predict the output of `print(17 // 5, 17 % 5)`.
6. Correct: `if mark >= 50 print("Pass")`.
7. Write a Python program to calculate simple interest.
8. Write a Python program to classify a number as positive, negative or zero.
9. Write a loop to print the multiplication table of a number.
10. Explain how an infinite while loop occurs and how to prevent it.

M4.04 • 6P • Continuous Internal Evaluation

Open-ended Project Handbook

Projects are compulsory for continuous evaluation, completed by groups of two or three, and should not be duplicated between groups.

Project development cycle

1. Select a useful problem and obtain topic approval.
2. Write objectives, expected output and task allocation.
3. Create a project folder and naming standard.
4. Collect only necessary data and use demonstration data where privacy matters.
5. Build the document, spreadsheet, presentation or Python component.
6. Test normal, boundary and invalid cases.
7. Capture meaningful screenshots and observations.
8. Prepare report, presentation and final files.
9. Rehearse demonstration and viva.
10. Submit editable files, PDF and backup as instructed.

Project tip: Group member □□□□□□□□□□□ □□□□□□□ contribution explain □□□□□□□ □□□□□□. Copy □□□□□ output □□□□□□ submit □□□□□□□□□.

Project 1 — Ten-page report

Prepare cover page, certificate, acknowledgement, table of contents, chapters, images, tables, captions, header/footer, page numbers, conclusion and references. Export PDF and email it as an attachment.

```
ProjectReport/
├── source/
├── images/
```

```
├── data/
├── export/
└── backup/
```

Suggested report topics

- Computer laboratory safety
- Digital privacy for students
- Office tools in education
- Evolution of computer storage
- Internet search and misinformation

Project 2 — Course presentation

Prepare slides for hardware, OS, Internet, Writer, Calc, Impress, algorithms, flowcharts and Python. Include relevant images, one table, one chart and permitted audio/video. Use a consistent theme and limited animation.

Slide plan

1. Title and team
2. Course objectives
3. Computer and ports
4. OS and file management
5. Internet and email
6. Office applications
7. Algorithm/flowchart
8. Python examples
9. Project work
10. Summary and references

Project 3 — Class rank list

Create student data, subject marks, total, average, percentage, pass/fail and rank. Sort correctly, apply conditional formatting and create a meaningful chart. Verify at least three records manually.

> **Minimum formula evidence:** SUM, AVERAGE, IF, percentage and rank using an absolute range.

Use demonstration names and marks unless the faculty authorises real class data.

Project 4 — Electricity bill or payroll

Electricity bill: consumer details, units, slab charge, fixed charge and total. Test 0, 100, 101, 200 and 201 units.

Payroll: employee details, basic pay, HRA, DA, other allowance, deductions, gross and net pay. Store common rates in cells and use absolute references.

> **Validation:** Negative units, negative pay and impossible values must be rejected or clearly flagged.

Project report and submission checklist

Planning

- ✓ Approved topic
- ✓ Clear objective
- ✓ Group roles
- ✓ Folder structure
- ✓ Timeline

Implementation

- ✓ Correct formulas/code
- ✓ Readable design
- ✓ Source files
- ✓ Meaningful screenshots
- ✓ Test cases

Submission

- ✓ Editable originals
- ✓ Verified PDF
- ✓ Presentation
- ✓ Backup
- ✓ Professional email

Area	Excellent evidence	Needs improvement
Planning	Clear objective, roles and folder structure	Work begins without a plan
Technical correctness	Correct content, formulas and program logic	Unverified results
Tool use	Appropriate features used consistently	Manual work where automation is needed
Testing	Normal, boundary and invalid cases documented	Only one test
Documentation	Readable report with evidence and references	Missing screenshots or result
Teamwork	Roles and contributions are visible	Unequal or unclear contribution
Presentation/viva	Clear demonstration and explanation	Unable to explain own work
Submission	Correct names, formats and backup	Wrong or missing files

This is a suggested learning rubric, not a claim about an official mark distribution.

Workbook and practical examination

Practice Lab and Troubleshooting

Independent tasks, hints and realistic fault scenarios across the complete course.

Unsolved practice tasks

Computer and operating system — 15 tasks

1. Prepare a hardware inventory of one laboratory computer.
2. Classify twelve devices.
3. Draw and label a computer block diagram.
4. Match ten cables to ports.
5. Compare HDD and SSD.
6. Record system information.
7. Create the prescribed course folder tree.
8. Copy, move and rename sample files.
9. Search for files by extension.
10. Compress and extract a folder.
11. Create a shortcut.
12. Delete and restore a file.
13. Use five directory commands.
14. Explain absolute and relative paths.
15. Prepare a backup checklist.

Internet and email — 10 tasks

1. Identify parts of a browser window.
2. Perform an exact-phrase search.
3. Use a site-restricted search.
4. Compare two sources for credibility.
5. Bookmark an educational page.
6. Download and organise a PDF.
7. Draft a professional email.
8. Explain CC and BCC using scenarios.
9. Identify phishing signs.
10. Write a shared-computer logout checklist.

Writer — 10 tasks

1. Formal letter
2. Student profile
3. Two-column newsletter
4. Table with merged heading
5. Image and caption
6. Header/footer and page numbers
7. Automatic TOC
8. Find and replace
9. Styles-based report
10. Export and verify PDF

Calc — 10 tasks

1. Basic arithmetic worksheet
2. Mark sheet
3. Percentage and result
4. Rank list
5. Expense sheet
6. Electricity bill
7. Payroll
8. Absolute-reference tax/discount
9. Sort and filter
10. Chart

Impress and cloud — 16 tasks

1. Five-slide hardware presentation
2. Course presentation
3. Use a slide master
4. Insert a table
5. Insert a chart
6. Embed media
7. Add speaker notes
8. Print handouts
9. Create a cloud folder
10. Upload files
11. Share as Viewer
12. Share as Commenter
13. Collaborate as Editor
14. Use comments
15. Inspect version history
16. Export/download and remove sharing

Algorithms and flowcharts — 15 problems

1. Area of a circle
2. Simple interest
3. Average of five values
4. Even or odd
5. Largest of three
6. Pass or fail
7. Grade
8. Leap year
9. Discount
10. Print 1–n
11. Sum 1–n
12. Factorial
13. Reverse number
14. Prime test
15. Fibonacci

Python — 25 challenge variations

1. Age in months
2. Currency conversion using a supplied rate
3. Body-mass-index calculator using demonstration values
4. Profit or loss
5. Quadratic discriminant classification
6. Admission eligibility
7. Telephone bill slabs
8. Income bonus rule
9. Vowel/consonant
10. Days in month
11. Sum of even numbers
12. Count positive inputs
13. Sentinel-controlled average
14. Power using a loop
15. LCM
16. Perfect number
17. Palindrome string
18. Number pattern
19. Menu calculator loop
20. Input validation loop
21. Multiplication tables 1–10
22. Prime numbers in a range
23. Armstrong numbers in a range
24. Repeated bill calculator
25. Project data validation

Troubleshooting handbook

Symptom	Likely reason	Diagnostic step	Correction / prevention
Computer does not start	Power, UPS, cable or hardware issue	Check indicators and authorised connections	Use correct power sequence; report hardware fault
Monitor shows no signal	Wrong input, loose cable or sleeping system	Check source and video cable	Reconnect gently and select correct input
Keyboard/mouse not detected	Port, battery, pairing or driver issue	Check connection and authorised alternate device	Reconnect or replace battery; report driver fault
USB not recognised	Damaged port/device or unsafe media	Try authorised alternate port/device	Do not force; scan media; report failure
No network	Cable, Wi-Fi, airplane mode or configuration	Check link icon and connection state	Reconnect or request support
No audio	Muted output, wrong device or port	Check volume and selected output	Select correct output and cable
File cannot be found	Wrong folder/name/extension	Search by partial name and inspect path	Use meaningful names and organised folders
Permission denied	Insufficient rights or protected location	Check owner and location	Use permitted folder; do not bypass controls
Command not recognised	Wrong command or typing error	Check OS and syntax	Use correct command/help
Incorrect path	Current directory differs	Use `pwd`/`cd` and list contents	Navigate or use a correct absolute path
Page not loading	Connection, URL or server issue	Check domain and another permitted page	Correct URL, retry or report issue
Download blocked	Browser/security policy	Inspect warning and source	Download only approved files
Attachment missing	File not attached or upload incomplete	Inspect message before sending	Attach first and verify preview
Suspicious email	Phishing	Inspect sender/domain and context	Do not click; report it
Image moves in Writer	Anchor or wrap setting	Inspect anchor/wrap	Choose correct anchoring and alignment
Page number missing	Typed text instead of field or wrong page style	Inspect footer and fields	Insert page-number field in correct style
Calc formula error	Wrong separator, reference or data type	Inspect formula bar and source cells	Correct formula and test manually
Chart shows wrong data	Incorrect range or labels	Highlight source range	Select correct categories and series
Presentation media fails	Unsupported format or broken link	Test media separately and on target computer	Use supported format and project folder
Wrong cloud permission	Sharing too broad or narrow	Open sharing settings	Apply least required permission
SyntaxError	Grammar error, often missing colon/bracket	Read the error line and caret	Correct syntax
IndentationError	Inconsistent indentation	Inspect block alignment	Use consistent spaces
NameError	Variable undefined or misspelled	Compare spelling and assignment order	Define before use
ValueError	Invalid conversion	Inspect input text	Validate before conversion
Division by zero	Divisor is zero	Test divisor	Reject zero before division
Infinite loop	Condition never becomes false	Trace loop state	Initialise and update correctly

Symptom	Likely reason	Diagnostic step	Correction / prevention
Logical error	Wrong formula or boundary	Dry-run known test cases	Correct algorithm and retest boundaries

Last-day revision

Quick Revision Sheet

Compact memory map for practical work, viva and series tests.

Computer cycle
Input → Process → Output ↔ Storage.

RAM vs storage
RAM temporary; SSD/HDD persistent.

HDMI
Digital video and audio.

RJ45
Wired Ethernet.

Copy vs move
Duplicate vs change location.

Path
Location of file/folder.

HTTPS
Encrypted transport; not automatic proof of trust.

BCC
Hides recipient list.

Writer styles
Consistent headings and automatic TOC.

Calc formula
Begins with =.

$B$2
Absolute reference.

Impress
One main idea per slide.

Cloud permission
Viewer, Commenter, Editor.

Algorithm
Finite, precise ordered steps.

Decision
Diamond with labelled branches.

input()
Returns a string.

// and %
Floor quotient and remainder.

if
Selection.

for
Known sequence/count.

while
Condition-controlled repetition.

Revision order: Hardware and ports → files/commands → Internet/email → Writer → Calc → Impress/cloud → flowchart symbols → Python operators → selection → loops → projects.

□□□□□ □□□□□: Definitions □□□□□□ □□□□□□□□□□ one Calc formula, one flowchart, one if program, one loop program dry run □□□□□□□.

Practical viva

Sixty Viva Questions and Answers

Concise answers plus practical demonstration prompts.

1. What is hardware?

Physical components of a computer.

2. What is software?

Programs and instructions used by a computer.

3. Role of CPU?

It executes instructions and controls processing.

4. RAM versus ROM?

RAM is temporary working memory; ROM is non-volatile memory for fixed instructions.

5. HDD versus SSD?

HDD uses magnetic moving parts; SSD uses flash memory and is usually faster.

6. What is a peripheral?

A device connected to a computer, such as a printer.

7. Why use UPS?

For short backup power and surge protection.

8. What is HDMI?

A digital audio/video interface.

9. What is RJ45 used for?

Commonly wired Ethernet networking.

10. Why not force a connector?

Wrong orientation or type can damage pins and ports.

11. What is an operating system?

System software managing hardware and services.

12. What is a file path?

The location of a file or folder.

13. Copy versus move?

Copy duplicates; move changes location.

14. What is a file extension?

A suffix indicating file type or association.

15. What is a relative path?

A path interpreted from the current working directory.

16. What does mkdir do?

Creates a directory.

17. What does cd do?

Changes the current directory.

18. Why check path before delete?

To avoid deleting the wrong data.

19. What is a browser?

An application for accessing web resources.

20. Browser versus search engine?

Browser is software; search engine is a web service.

21. What is a URL?

The address of a web resource.

22. HTTP versus HTTPS?

HTTPS encrypts HTTP transport using TLS.

23. What is phishing?

Fraud attempting to steal credentials or data.

24. What is BCC?

A hidden copy to recipients.

25. Why verify an attachment?

To ensure the correct file is sent.

26. What is Writer used for?

Creating and formatting documents.

27. Why use styles?

For consistency and automatic features such as TOC.

28. What is a page break?

A controlled start on a new page.

29. Why add captions?

To identify and reference figures/tables.

30. Save versus Save As?

Save updates current file; Save As creates another file/name/location/format.

31. What is a cell?

Intersection of a row and column.

32. What is a cell address?

Column letter plus row number, such as B4.

33. What is a formula?

An expression beginning with = that calculates a value.

34. Relative versus absolute reference?

Relative changes when copied; absolute remains fixed.

35. What does SUM do?

Adds numeric values in a range.

36. What does IF do?

Returns a result based on a condition.

37. Why verify a spreadsheet manually?

To detect wrong references or logic.

38. What is a slide master?

A template controlling common slide design.

39. Why limit animation?

Too much animation distracts from content.

40. Why test media?

Format or file links may fail elsewhere.

41. What is cloud storage?

Remote storage accessed through an account and network.

42. Viewer versus Editor?

Viewer reads; Editor can change content.

43. What is version history?

A record of previous edits that can be inspected or restored.

44. Why use least permission?

To reduce accidental or unauthorised changes.

45. What is an algorithm?

A finite, precise sequence of solution steps.

46. What is pseudocode?

Structured language-independent algorithm description.

47. What does a decision diamond show?

A condition and alternative branches.

48. What is a dry run?

Manual execution with test data.

49. What is sequence?

Execution of statements in order.

50. What is iteration?

Repeated execution of steps.

51. What does input() return?

A string.

52. Why use int() or float()?

To convert input text for numeric calculation.

53. What is indentation?

Whitespace defining a Python block.

54. / versus //?

/ gives true division; // gives floor division.

55. What does % return?

Remainder.

56. What is a Boolean expression?

An expression resulting in True or False.

57. When is for preferred?

For a sequence or known count.

58. When is while preferred?

When repetition depends on a condition.

59. What causes an infinite loop?

The condition never becomes false, often due to missing update.

60. What must a project include?

Planning, implementation, testing, evidence, report, presentation and viva explanation.

Practical demonstration viva

1. Identify HDMI, USB and Ethernet ports.
2. Create and rename a folder.
3. Show the current path in a terminal.
4. Perform an exact-phrase search.
5. Draft an email with attachment.
6. Apply a heading style and insert a TOC.
7. Enter a SUM formula and copy it.
8. Change a cloud permission from Editor to Viewer.
9. Draw an even/odd flowchart.
10. Run and modify a Python loop.

Master Answer Key

Answers, Marking Points and Practice Assessment

Complete guidance for the original series tests and integrated practical preparation.

Series Test I answers

1. **Volatile:** RAM. **Non-volatile:** ROM, SSD or HDD.
2. Copy creates a duplicate while preserving the original; move changes location.
3. HDMI carries digital audio/video; RJ45 commonly connects Ethernet.
4. Phishing signs include a mismatched domain, urgent credential request, suspicious link/attachment and unexpected context.
5. Save updates current file; Save As creates another file/name/location/format; Export produces an output such as PDF.
6. Use the folder tree in Module 1 and provide screenshot/command evidence.
7. A letter needs addresses, date, subject, salutation, clear body, closing and verified PDF export.
8. Use `SUM`, `AVERAGE` and `IF`; manually verify at least one row.
9. Possible slides: title, risks, strong passwords, phishing, safe downloads/logout.
10. Viewer reads, Commenter gives feedback, Editor changes content.

Series Test II answers

1. An algorithm is a finite, definite and effective ordered sequence that accepts input and produces output.
2. Terminator, process, input/output and decision symbols are shown in Module 3.
3.
```
READ a, b
IF a > b THEN PRINT a
ELSE PRINT b
END IF
```
4. 572: total 0 → 2 → 9 → 14, so the digit sum is 14.
5. `17 // 5` is 3 and `17 % 5` is 2.
6.
```
if mark >= 50:
    print("Pass")
```
7. Use the simple-interest program in Module 4 and explain conversion to float.
8. Use the three-way selection program and validate input where required.
9.
```
number = int(input("Number: "))
for i in range(1, 11):
    print(number, "x", i, "=", number * i)
```

10. An infinite loop occurs when the condition remains true; initialise correctly and update the loop variable/state.

Original syllabus-based practice assessment

Not an official board question paper. Use it for integrated practical preparation.

Task A — Computer and OS

Identify five components and five ports, then create the prescribed course folder structure using GUI and at least three commands.

Task B — Internet and email

Find a credible Python reference, record evaluation criteria and draft a professional email with a sample attachment.

Task C — Office tools

Create a two-page Writer report, a Calc mark sheet and a five-slide Impress summary. Export outputs and verify them.

Task D — Algorithm and Python

Design an algorithm and flowchart, then write Python for an electricity bill or payroll problem. Test boundaries.

Task E — Viva

Explain file paths, cloud permissions, spreadsheet references, decision flowcharts and loop termination.

Evaluation area	Expected evidence
Correctness	Required result, formulas and program logic are correct
Procedure	Steps are logical and safe
Testing	Normal and boundary cases are documented
Presentation	Files are organised and output is readable
Explanation	Student can explain decisions and corrections

Practical record templates

Computer practical

Experiment no. • Date • Aim • Requirements • Theory • Procedure • Observation/screenshot • Result • Precautions • Signature.

Algorithm practical

Problem • Input • Output • Algorithm • Pseudocode • Flowchart • Dry run • Result.

Python practical

Problem • Algorithm • Program • Test input • Output • Additional test cases • Result.

References from the syllabus

1. Rajaraman V. — *Fundamentals of Computers*, PHI.
2. Mrs. Chetna Shah and Mr. Kalpesh Patel — *Open Office*.
3. E. Balagurusamy — *Introduction to Computing and Problem Solving Using Python*, McGraw Hill.

Official online-resource topics include computer fundamentals, Python learning, free tutorials and the official Python website.